RESEARCH PROPOSAL

Autonomous Object Navigation System using Elastic Distributed Inference with Video Vision Transformer and CNN Architectures on Dynamic Edge Device Pools

Shahmeer Hyat
Muhammad Soban
Ali Rauf

1. Problem Definition

1.1 Task Description

The input to the model is a real-time RGB frame obtained from a camera on an edge device (in our case a simulated world robot) which resized to a fixed spatial resolution. Given this input the system must produce an output action command such as move forward, turn left, or stop that directs the robot towards the detected target object.

1.2 Motivation and Relevance

1.2.1 Why Is This Problem Important?

Edge devices such as the Nvidia Jetson and mobile robots have limited compute power, meaning that running all inference locally risks overheating, significant FPS drops, reduced model accuracy due to quantization, and increased energy consumption. On the other hand, offloading all computation to a remote server introduces high network latency, bandwidth costs, and vulnerability to network instability, all of which are unacceptable in a real-time autonomous navigation context.

Prior split inference frameworks such as SPViT address this by partitioning model computation between a fixed, predetermined set of devices. However, this tight coupling to a static device topology creates a fundamental brittleness which is that if any participating device becomes unavailable, the entire inference pipeline is disrupted. In real-world deployments if edge nodes fail, join, and/or leave dynamically and a system that cannot adapt to this reality is impractical.

Our solution is elastic split inference: a system that dynamically discovers available edge nodes at runtime, distributes ViT layer execution across however many devices are currently alive, and gracefully degrades as nodes drop out, ultimately falling back to cloud execution when no edge

resources remain. For example, inference may begin across five available edge nodes; if three go offline, the remaining two absorb the workload; if all edge nodes disappear, the full computation migrates to the cloud without interruption. This elastic approach has direct applications in autonomous robotics, real-time object tracking, swarm robotics, and IoT-based surveillance systems where device availability is inherently unpredictable.

1.2.2 Why Deep Learning?

Raw images are high dimensional signals where traditional hand crafted features such as SIFT and HOG fail under noisy conditions. Deep neural networks can address this by automatically learning hierarchical & noise-invariant feature representations directly from data, removing the need for manual feature engineering. Vision Transformers (ViT) are particularly well suited to this problem primarily for three reasons:

- They model global spatial relationships across the entire image simultaneously via the self-attention mechanism.
- They are modular by design, making them straightforward to split layer-wise for distributed execution across a variable number of devices.
- Their intermediate token representations are semantically meaningful, allowing computation to be cleanly partitioned at any layer boundary without losing representational coherence, which is essential for dynamic re-partitioning as device availability changes.

Furthermore we are also going to use YOLO models which will be executed solely on the edge device which will be used to track basic objects which are semantically important for the system to understand.

1.2.3 The Curse of Dimensionality

A raw 640 by 480 by 3 image contains 921k pixel values. This dimensionality causes data sparsity, poor generalization in shallow models, and makes manual feature engineering practically infeasible. Deep learning solves this problem by learning latent representations that convert the high-dimensional raw input to a low dimensional embedding space, improving generalization and enabling the model to capture the underlying structure of the data.

1.3 Research Questions

The following research questions guide this investigation:

Primary Research Question: Can a dynamically partitioned and elastically distributed split Vision Transformer architecture maintain comparable accuracy to a fully centralized model while also significantly reducing edge computation and network bandwidth even as the number of participating devices are fluctuating at runtime?

RQ1 Optimal Split Depth: At which Transformer layer should the model be partitioned in order to achieve the best trade off between inference latency & also classification accuracy?

RQ2 Noise Robustness: How does noisy input affect the performance of split inference compared to fully local or fully remote inference?

RQ3 Quantization Strategy: Does quantizing the YOLO model impact final prediction accuracy?

RQ4 Dynamic Topology Resilience: How does inference accuracy and latency degrade as participating edge nodes drop out at runtime, and at what device pool size does cloud fallback become the superior strategy?

2. Literature Review

2.1 Overview

The field of video-based perception for autonomous robotics has undergone a dramatic transformation over the past decade, mirroring the broader trajectory of deep learning described by Goodfellow et al. (2016): from hand-crafted features and shallow classifiers to deep convolutional networks, to the current generation of Transformer-based architectures that model global dependencies across space and time.

The central goal of this work is to develop an autonomous robot capable of detecting and continuously following a target object in real time. This requires a perception system that is not only accurate but also responsive to the computational constraints of embedded edge hardware such as the Raspberry Pi. To address this, we propose a dynamic inference strategy: when the task complexity is low such as detecting a clearly visible target under stable lighting a lightweight CNN model runs entirely on-device, minimizing latency. When the scene is more demanding involving occlusion, multiple objects, or ambiguous poses a Vision Transformer (ViT) is invoked for its superior global context modelling, with its inference workload split across a dynamically discovered pool of available edge devices and a remote cloud server.

Early deep learning approaches adapted image classification CNNs to real-time perception tasks by processing individual frames efficiently, leveraging strong spatial inductive biases to detect and localize objects with low overhead. More recently, the introduction of ViT by Dosovitskiy et al. (2021) demonstrated that pure attention-based models could match or exceed CNN performance when trained at scale, and collaborative inference frameworks such as SPViT (Zhao et al., 2025) have since shown that ViT inference can be partitioned across heterogeneous edge devices to make deployment feasible. However, SPViT and similar frameworks assume a fixed, tightly coupled device topology, leaving the open problem of elastic, fault-tolerant distributed inference across a dynamically changing device pool. The current frontier, and the direction this work occupies, involves not only dynamically selecting between CNN and split ViT inference at runtime, but also dynamically discovering and reassigning computation across a variable number of live edge nodes, with seamless fallback to the cloud as devices drop out.

2.2 Detailed Review of Baseline Papers (2024–2026)

Baseline 1: ***Zhao, Liu, Jin, and Yao (2025)*** SPViT: Adaptive Splitting and Offloading for Mobile ViT Inference

Problem and Data: Zhao et al. (2025) address the problem of deploying Vision Transformer models on mobile and edge devices, where computational resources (CPU, memory, battery) are severely constrained. Rather than treating this as a standalone classification problem, they frame it as an inference efficiency problem: given that ViT models are too large for a single mobile device, how can inference be partitioned and distributed across multiple nearby devices collaboratively? They evaluate on ImageNet-22K and a real-world prototype smart home scenario, measuring inference latency, FLOPs, and communication cost.

Deep Model Idea: SPViT introduces a method called adaptive splitting and offloading. The ViT model is divided into segments at carefully chosen split points, and different segments are executed on different devices (smartphones, tablets, smartwatches) in parallel or in sequence. The authors introduce two novel algorithms ARIMA-V and Adaptive ARIMA-V for predicting network bandwidth conditions and dynamically selecting the optimal split strategy in real time. They also propose novel "Head-Width" and "Neuron-Width" splitting techniques that partition the internal dimensions of Transformer layers rather than simply cutting between layers. The system is trained with standard gradient-based methods; the splitting logic is added at inference time without retraining.

Results and Metrics: SPViT is benchmarked against DEViT and ED-ViT, two prior collaborative inference frameworks. It achieves significantly lower inference latency and communication overhead while maintaining competitive accuracy. Code is publicly available.

Strengths: Published in IEEE Transactions on Mobile Computing (Rank A journal), this paper provides a principled, theoretically grounded approach to a practically critical problem. The adaptive splitting strategy is directly relevant to real-time deployment on edge clusters.

Weaknesses / Gaps: SPViT assumes a fixed, tightly coupled device topology: the number of participating devices is specified in advance, ARIMA-V optimizes bandwidth prediction for a predetermined set of endpoints, and there is no mechanism for handling node failure or dynamic join/leave of devices mid-inference. If a participating device goes offline, the inference pipeline breaks. This represents a fundamental limitation for real-world deployment scenarios where device availability is unpredictable. Our proposed work directly addresses this gap by replacing the static device topology assumption with an elastic, loosely coupled architecture in which the system discovers available nodes at runtime via lightweight heartbeat signalling, dynamically reassigns ViT layer partitions as nodes join or leave, and gracefully degrades to cloud execution when edge resources are exhausted.

Baseline 2: ***Rosero-Montalvo, Tozun, and Hernandez (2024)*** Optimized CNN Architectures on Edge Devices for IoT

Problem and Data: Rosero-Montalvo et al. (2024) study the challenge of deploying deep learning models on hardware-constrained edge devices in IoT environments. In many IoT

applications, such as smart monitoring or real-time perception systems, deep neural networks must run on devices with limited computational power, memory, and energy resources. To address this problem, the authors evaluate how different lightweight convolutional neural network (CNN) architectures perform when deployed on edge hardware. Instead of proposing a new dataset, the work focuses on benchmarking pretrained CNN models using transfer learning and measuring how efficiently they run on constrained devices.

Deep Model Idea: The study compares several compact CNN architectures that are commonly used for efficient inference on edge systems. These architectures include lightweight models designed to reduce computation while maintaining acceptable accuracy. The models are initialized using pretrained weights and then fine-tuned through transfer learning. By benchmarking these architectures, the authors aim to identify which CNN designs provide the best trade-off between accuracy and computational efficiency when deployed in IoT environments.

Results and Metrics: The evaluation focuses mainly on performance metrics relevant to edge deployment, including inference time, energy consumption, memory usage, and model accuracy. The results show that lightweight CNN architectures can achieve reasonable accuracy while significantly reducing computational requirements, making them suitable for real-time inference on edge devices. However, the paper mainly presents benchmarking results rather than proposing a new optimized architecture.

Strengths: One major strength of the work is that it provides a practical evaluation of CNN architectures on real edge hardware, which is important for IoT applications where resources are limited. Another advantage is the focus on deployment metrics such as latency and power consumption, which are often overlooked in purely academic deep learning studies. The paper helps developers understand which CNN models are more suitable for resource-constrained environments.

Weaknesses / Gaps: A limitation of the study is that it mainly performs benchmarking rather than proposing a new model or algorithm. As a result, the contribution is more focused on evaluation than innovation. Additionally, the paper focuses on image classification tasks and does not address more complex perception tasks such as object detection, pose estimation, or action recognition. For applications like Human Action Recognition (HAR), which require temporal understanding of human activities, further modeling approaches would be needed.

2.3 Additional Related Papers

Paper 3: *Dosovitskiy et al. (2021)* An Image is Worth 16x16 Words (ViT)

This paper basically introduced the Vision Transformer (ViT) idea for computer vision. Instead of using convolutions like CNNs normally do, the authors treat an image as a sequence of smaller patches and process them using a Transformer encoder, similar to how transformers process tokens in NLP. The image is first divided into 16x16 patches, each patch is then linearly embedded into a vector and positional encodings are added so the model still knows the spatial

order of the patches. After that the sequence is passed through the transformer layers. The authors showed that when the model is pretrained on really large datasets like JFT-300M or ImageNet-21K, it can actually outperform strong CNN models such as ResNet and EfficientNet. But on smaller datasets it usually does worse, mainly because it doesnt have the convolutional inductive biases that CNNs have built in. Overall this paper became very important since most of the modern vision transformer models are based on this same core idea.

Paper 4: *Arnab, Dehghani, Heigold, Sun, Lucic, and Zisserman (2021)* ViT: A Video Vision Transformer

ViT extended the ViT paradigm to video classification, proposing four factorized model variants to handle the combinatorial explosion of spatiotemporal tokens that arise when treating a video as a 3D grid of patches. The key contribution is the "tubelet embedding" extracting short spatio-temporal tubes from the video volume rather than individual frame patches combined with factorized attention that models spatial and temporal dimensions separately for efficiency. ViT achieved state-of-the-art results on Kinetics-400, Kinetics-600, Epic Kitchens 100, Something-Something v2, and Moments in Time, outperforming prior 3D CNN methods such as I3D and SlowFast. For the present proposal, ViT is the conceptual backbone for video understanding with Transformers: its tubelet embedding and factorized attention design principles directly inform how we will represent video inputs and structure temporal modelling in our proposed model.

Paper 5: *Han and Huang (2024)* ACTrack: Adding Spatio-Temporal Condition for Visual Object Tracking

ACTrack addresses the problem of visual object tracking following a designated target through consecutive video frames by introducing a lightweight additive siamese convolutional network that is trained on top of a frozen pretrained Transformer backbone. Rather than fine-tuning the entire Transformer (which is computationally expensive), ACTrack trains only a small spatio-temporal conditioning network that captures motion context between consecutive frames. This design philosophy freezing large pretrained backbones and training only efficient adapter modules is directly relevant to our proposed approach, where we aim to leverage pretrained ViT weights and train lightweight temporal fusion modules for HAR without incurring the full cost of end-to-end Transformer training. ACTrack is evaluated on LaSOT, GOT-10k, TrackingNet, and TNL2K using AUC, Precision, and Normalized Precision metrics.

Paper 6: *Li, Wu, Du, Liu, Nghiem, and Shi (2025)* A Survey of Large Vision Language Models

This paper is a survey about Large Vision Language Models (LVLMs). It mainly looks at how modern models combine visual understanding with language reasoning. The survey discusses different architectures, alignment techniques, pretraining strategies and evaluation benchmarks for models like GPT-4V, Gemini and Claude-3. Even though the paper is not directly about action recognition or edge systems, it still gives a good idea of where vision research is heading,

which is towards large multimodal models that can reason about images and text together. It also talks about techniques like instruction tuning and RLHF for vision models, which are becoming common for aligning these models with human instructions.

Paper 7: *Alomar, Aysel, and Cai (2025)* CNNs, RNNs and Transformers in Human Action Recognition: A Survey and a Hybrid Model

This paper provides a survey of different architectures used in Human Action Recognition (HAR) and also proposes a hybrid model. The survey explains how the field moved from CNN based models to RNNs and now to transformer based approaches. Their proposed model combines CNNs for extracting spatial features from frames and a Vision Transformer encoder for learning temporal relationships across the video frames. The model is tested on datasets like UCF101, HMDB51, Kinetics-400 and KTH and shows better top-1 accuracy compared to pure CNN or pure ViT models. One useful takeaway from this paper is the idea that combining CNN spatial features with transformer based temporal modelling can give better results. However the work mainly focuses on accuracy and does not really address deployment issues like edge hardware or split inference, and critically it does not consider scenarios where the set of available compute devices changes dynamically during inference, which is central to our proposed system.

3. Proposed Deep Learning Approach

3.1 High-Level Idea

We propose a supervised deep learning system for real-time autonomous target following on an edge device (in our case simulation). The system takes as input a live RGB camera frame and produces an action command such as move forward, turn left, or stop that steers the robot towards the detected target. Rather than using a single fixed architecture, we propose a dynamic inference strategy that selects between two models at runtime based on object type and current device availability.

When the object is simple such as a car or ball, the task is sent to a lightweight CNN YOLO model which runs entirely on the edge device, minimizing latency and energy consumption. When the prompt is more demanding or requires richer spatial context, a Vision Transformer is invoked for its superior global reasoning capacity. Crucially, the ViT inference is not split across a fixed, predetermined set of devices as in SPViT. Instead, the system maintains an elastic device pool: a coordinator process continuously broadcasts lightweight heartbeat messages to discover which edge nodes are currently reachable. The available nodes at any given moment form the active pool, and ViT layers are partitioned dynamically across however many nodes are alive.

If nodes drop out during or between inference cycles, the coordinator redistributes the layer assignments across the remaining live nodes. If the active pool shrinks to zero, the full ViT computation migrates seamlessly to the cloud server. This means the system degrades gracefully rather than failing catastrophically, moving from five-node distributed inference to three-node to

one-node to cloud-only execution as device availability decreases, all without requiring a restart or manual reconfiguration.

The YOLO model is trained under the supervised learning paradigm using cross-entropy loss and gradient-based optimization (Adam), learning an idea of how noise occurs in the simulation world.

3.2 Input Representation

The input to the model is a real-time RGB frame obtained from a camera on an edge device (in our case a robot in simulation world), resized to a fixed spatial resolution. Pixel values are normalized to zero mean and unit variance using ImageNet statistics, ensuring compatibility with pretrained model weights.

3.3 Network Outline

The system consists of three components: two inference pathways and an elastic coordination layer.

Lightweight YOLO CNN Pathway (On-Device): A YOLO model processes the input frame entirely on the edge device. The model is fine-tuned for target detection in a noisy environment, following the transfer learning strategy demonstrated by Rosero-Montalvo et al. (2024). It is fast, energy-efficient, and does not require any network communication.

Elastic Split ViT Pathway (Dynamic Edge Pool + Cloud): When the scene gets too complex for the CNN to handle reliably, the frame is forwarded to the Vision Transformer pathway. The ViT first splits the image into fixed-size patches, linearly embeds them, and processes them through several Transformer encoder layers using self-attention. The model is split such that each segment of consecutive encoder layers is assigned to a different node in the currently active device pool. A lightweight coordinator manages this assignment. It maintains a live registry of available nodes through periodic heartbeat polling, assigns encoder layer blocks to nodes proportional to their reported compute capacity, and monitors for node dropout. If a node goes silent, its assigned layers are immediately redistributed among the remaining live nodes. If all edge nodes are unavailable, the full ViT computation is forwarded to the cloud server. The intermediate token representations passed between nodes are semantically meaningful as demonstrated by Zhao et al. (2025), which justifies their transmission across node boundaries without loss of representational coherence. Unlike SPViT, the number and identity of participating nodes is never fixed in advance and requires no static configuration.

Dynamic Coordinator: A lightweight master process runs on the primary edge device or a dedicated orchestration node. It performs three functions: (1) device discovery through periodic heartbeat broadcasting over the local network, (2) layer assignment by partitioning the ViT encoder into blocks and assigning blocks to live nodes based on current availability and estimated capacity, and (3) fault recovery by detecting unresponsive nodes and triggering reassignment of their blocks to remaining live nodes or to the cloud fallback. This coordinator is

the architectural element that distinguishes our system from tightly coupled frameworks like SPViT.

3.4 Why This Approach is Suitable

As Goodfellow et al. (2016) argue, deep networks are most appropriate when data is high-dimensional and the target function is complex, precisely the characteristics of real-time visual perception for robotics. The CNN pathway embodies efficient representation learning for the common case, automatically discovering spatially discriminative features without hand-crafted engineering. The ViT pathway addresses the harder case by leveraging global self-attention to reason about the entire scene simultaneously, capturing spatial relationships that local convolutions miss, as validated by the JAICS (2024) navigation framework.

The elastic split inference design is further justified by both the modular structure of Transformers and the practical realities of edge deployments. Because intermediate token representations carry meaningful semantic content, computation can be cleanly partitioned at any layer boundary and reassigned to different nodes at runtime without loss of representational coherence. This directly enables the dynamic execution strategy central to this proposal. Unlike SPViT which optimizes for a fixed topology, our approach treats the device pool as a first-class variable, enabling deployment in environments where device availability is inherently non-deterministic.

4. Learning and Understanding Outcomes

Beyond benchmark performance, this project is designed to deepen understanding of the following deep learning concepts:

Effect of split depth in ViT: By ablating the layer at which ViT is partitioned between edge devices and the server (early split vs. mid split vs. late split), we will empirically observe how the choice of split point affects both inference accuracy and end-to-end latency, directly connecting to RQ1 and the depth-generalization trade-off discussed by Goodfellow et al. (2016).

CNN vs. Transformer trade-off: Comparing the YOLO pathway against the full split ViT pathway across varying scene conditions will provide concrete empirical insight into when global self-attention genuinely outperforms local convolution and at what computational cost, grounding the theoretical complementarity argument in measurable real-world results.

Benefit of transfer learning: Comparing a pretrained CNN backbone (whether frozen or fine-tuned) with a randomly initialized backbone will show how ImageNet pretraining actually helps with the target detection task. This demonstrates that the features learned on a big dataset can be reused hierarchically for a new task, as discussed in the representation learning literature and shown by Rosero-Montalvo et al. (2024).

Hyperparameter sensitivity: Performing a grid search over learning rates, dropout rates, and batch sizes for both the CNN and ViT components will reveal how sensitive each model is to these choices, connecting to the practical training issues discussed by Goodfellow et al.

Dynamic topology effects: By deliberately varying the number of available edge nodes during inference experiments, we will empirically characterize how accuracy and latency degrade as the device pool shrinks, and identify the crossover point at which cloud fallback becomes the dominant strategy. This connects directly to RQ4 and provides insight into the costs and benefits of loose vs. tight coupling in distributed inference systems.

4.1 Potential Limitations

Edge device computational constraints: may limit how many ViT encoder layers can realistically run on-device before transmission, potentially forcing a very early split that increases communication overhead and reduces the benefit of partial edge processing.

Network variability: Fluctuating bandwidth, packet loss, or latency spikes between edge nodes and between the edge tier and cloud may significantly degrade the split ViT pathway's performance in ways that controlled lab experiments do not capture. The dynamic coordinator must be robust to these conditions, adding engineering complexity.

Coordinator overhead: The heartbeat-based discovery and layer-reassignment logic introduces additional latency on node dropout events. In scenarios with frequent node churn, this overhead could negate the benefits of elastic distribution. Characterizing and minimizing coordinator latency is a key engineering challenge.

Dataset Scarcity and Synthetic Domain Adaptation: A primary obstacle was the lack of specialized, publicly available datasets that mirror the specific visual characteristics of low-power robotic simulations. To overcome this, we performed domain adaptation on the COCO 2017 dataset. Because we could not find a dataset that natively represented our "Edge-to-Simulation" pipeline, we had to programmatically inject Gaussian noise, motion blur, and JPEG compression artifacts. This transformation was essential to ensure the YOLOv8 model could generalize from high-fidelity benchmark images to the degraded, low-resolution visual stream of the Webots environment.

5. References

Alomar, K., Aysel, H. I., & Cai, X. (2025). CNNs, RNNs and Transformers in human action recognition: a survey and a hybrid model. Artificial Intelligence Review, Springer Nature. [Q1 Journal]

Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lucic, M., & Zisserman, A. (2021). ViT: A Video Vision Transformer. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2021).

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. International Conference on Learning Representations (ICLR 2021).

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Han, Y., & Huang, K. (2024). ACTrack: Adding Spatio-Temporal Condition for Visual Object Tracking. arXiv:2403.07914. Lenovo Research.

Li, Z., Wu, X., Du, H., Liu, F., Nghiem, H., & Shi, G. (2025). A Survey of State of the Art Large Vision Language Models: Alignment, Benchmark, Evaluations and Challenges. CVPR Workshops (CVPRW 2025).

Rosero-Montalvo, P. D., Tozun, P., & Hernandez, W. (2024). Optimized CNN Architectures Benchmarking in Hardware-Constrained Edge Devices in IoT Environments. IEEE Internet of Things Journal / 25th IEEE International Conference on Industrial Technology.

Vision Transformers for Real-Time Autonomous Navigation in Unstructured Environments. (2024). Journal of Artificial Intelligence and Cyber Security (JAICS). [Open Access]

Zhao, S., Liu, T., Jin, H., & Yao, D. (2025). SPViT: Accelerate Vision Transformer Inference on Mobile Devices via Adaptive Splitting and Offloading. IEEE Transactions on Mobile Computing. [Rank A]